BayesianEstimationTool Manual

BayesianEstimationTool (BET) helps you fit a mechanistic model to a population of related individuals and then use what you learned to predict outcomes for a new individual. An *individual* (or *entry*) might be a patient, experiment, or replicate. You supply the scientific model and how observations arise; BET handles the repeated work of hierarchical Bayesian inference, population-to-individual transfer, and posterior prediction.

A mathematical model describes how a system changes or behaves. For example, a simple tumor growth model can describe how tumor size changes over time using a growth-rate parameter: a larger rate produces faster growth. If tumor sizes are measured repeatedly for several patients, the model can be compared with each patient's measurements to estimate how quickly their tumors grow. The same equations can describe every patient, even though the parameter values may differ.

Rather than estimating each patient's growth rate independently, BET uses a **hierarchical model** to learn how growth rates vary across the patient population. This allows information from patients with more measurements to inform estimates for patients with less data. For a new patient, BET can generate predictions from the learned population distribution before any tumor measurements are available, or use the patient's available measurements to update their individual growth-rate estimate. These are **population prediction** and **observation-conditioned prediction**, respectively. The distinction is central to BET's training and testing workflow.

The workflow at a glance

1. **Describe the data.** Store observations and covariates for each entry in arrays.
2. **Define the mechanistic model.** Write a function that predicts outputs for one entry from shared and individual parameters.
3. **Define uncertainty.** Supply priors for shared parameters, names for individual latent effects, and a likelihood describing measurement noise.
4. **Train.** Infer population parameters and the distribution of individual effects from training entries.
5. **Test.** Predict for unseen entries, optionally conditioning on their available observations.
6. **Score and inspect.** Compare predictions with held-out data and examine posterior uncertainty.

Core components

Component	Purpose	You provide
`data`	Inputs for all entries	Dictionary of JAX or NumPy arrays
`ModelFn`	Scientific model for one entry	Function returning named predictions
`GlobalPriorFn`	Prior distributions for shared unknowns	Function returning distributions
`localZNames`	Names of individual-varying standardized effects	List of strings
`TrainLikelihoodFn`	How training observations relate to predictions	Function returning observations and distributions
`Estimator`	Coordinates the model and inference state	Model, data, and optional prior/effect definitions
`Train` / `Test`	Population fitting / new-entry prediction	Indices, likelihood, and inference method

Component	Purpose	You provide
`ScoreFn`	Defines a prediction quality measure	Function returning a score per entry

Notation used throughout: `[]` denotes a list or array (for example, `[0, 1]`); `{}` denotes a dictionary mapping names to values (for example, `{"time": times}`); `(a, b)` denotes a tuple; and `[N, T]` describes an array shape with `N` entries and `T` time positions. These symbols indicate Python data structures, not additional BET syntax.

Part 1: Core concepts

Overview

BET separates the scientific model from hierarchical inference. You provide a deterministic model for one entry, optional priors for shared parameters, and an observation likelihood. BET handles population training, new-entry prediction, posterior sample bookkeeping, and scoring. It uses JAX to evaluate the model across entries and posterior draws, so model functions must use JAX-compatible operations and avoid side effects.

An **entry** is one independently indexed unit, such as a patient, experiment, or simulated replicate. Every data array has an entry axis first. Repeated observations within an entry occupy trailing axes and are modeled together by that entry's likelihood. Treating those observations as separate entries changes the hierarchical interpretation.

Shared parameters and local z-scores. Global parameters are shared across training entries; local latent effects vary by entry and have standard-normal marginal priors. For example, a model might define a positive rate as `rate_i = exp(log_rate_mean + log_rate_sd * z_i)`. Here `z_i` is a z-score on the log-rate scale, while `rate_i` is lognormally distributed on its physical scale. The location and scale parameters are supplied by the user model or inferred through `GlobalPriorFn`; BET does not automatically invent a mean and scale for every local variable.

With multiple local variables, a learned Cholesky factor can represent correlations among their standardized effects. Positive correlation means entries high in one effect tend to be high in another **before conditioning on individual observations**. Once conditioned on data, posterior local values need not have sample mean zero or standard deviation one.

Training and testing serve different purposes. Training uses selected entries to estimate shared parameters, correlations, and training-specific local effects. Testing uses the learned population state to predict outcomes for unseen entries. Without a test likelihood, BET samples local effects from the learned population distribution. With a test likelihood and inference Method, it conditions those effects on observations available for the new entry. For out-of-sample evaluation, exclude test entries from training; training itself estimates shared parameters, correlations, and training-specific local effects.

Designing a BET model

1. Prepare `data` as a dictionary of equally long first-axis arrays. Decide which trailing axes represent time, features, or spatial positions.
2. Implement `ModelFn(globalParams, localZs, data)` for **one entry**. Return named arrays or scalars in a dictionary. Do not sample NumPyro random variables in this deterministic model function.
3. Implement `GlobalPriorFn(dataShapes)` when shared parameters must be inferred. Return NumPyro distribution objects keyed by unique parameter names. If there are no shared unknown parameters, omit the function.
4. Specify `localZNames` only for genuine entry-varying effects. Map those standardized effects to physical parameters inside `ModelFn`.
5. Implement the training likelihood as a dictionary of `(observation, distribution)` pairs. Use distributions with correct event and batch shapes; use explicit masks for absent observations.

6. Select training indices and a configured `Nuts(...)` or `Svi(...)` instance. Call `Train`, inspect diagnostics, and verify model behavior with posterior predictive checks.
7. Select held-out indices. Call `Test` without a likelihood for population prediction, or provide a test likelihood and Method for inference using partial observations.
8. Define a score function returning a scalar or array for one entry and evaluate `ScoreTrainSamples` or `ScoreTestSamples`. Summarize over posterior axes deliberately; do not confuse draw axes with entry axes.

Keep the mechanistic model's parameters distinct from the observation process. Baseline covariates may enter `ModelFn` or transformations of global parameters; observation masks determine which outcomes contribute to inference.

Part 2: Worked example

Predicting longitudinal growth

This tutorial connects population training, predictions for unseen individuals, and updating those predictions after observing the beginning of each individual's trajectory. It uses synthetic exponential-growth data so that no external dataset is required. The goal is to compare **population-only prediction** against **prediction conditioned on three measurements**, using the same trained population model.

Generate data and define the model

```
import jax.numpy as jnp
from jax import random
import numpyro.distributions as dist
from BayesianEstimationTool import Estimator, Svi

n_people = 10
times = jnp.linspace(0.0, 7.0, 8)
key_g, key_noise = random.split(random.PRNGKey(42))
true_g = 0.15 + 0.025 * random.normal(key_g, (n_people,))
true_y = jnp.exp(true_g[:, None] * times[None, :])
y = true_y + 0.05 * random.normal(key_noise, true_y.shape)
data = {"time": jnp.tile(times, (n_people, 1)), "y": y}

def global_prior(shapes):
    return {
        "g_mean": dist.Normal(0.15, 0.10),
        "g_sd": dist.HalfNormal(0.10),
        "noise": dist.HalfNormal(0.10),
    }

def model_fn(global_params, local_zs, data):
    g = global_params["g_mean"] + global_params["g_sd"] * local_zs["g"]
    return {"prediction": jnp.exp(g * data["time"])}

def train_likelihood(global_params, local_zs, data, model_out):
    return {"y": (data["y"], dist.Normal(model_out["prediction"], global_params["noise"]))}
```

`g_mean` and `g_sd` set the population center and spread of growth rates. Each individual's standardized `local_zs["g"]` becomes a physical growth rate through `g_mean + g_sd * z`. The separate `noise` parameter describes measurement variation, not biological heterogeneity.

Train on six individuals

```
est = Estimator(data, model_fn, localZNames=["g"],
                GlobalPriorFn=global_prior, rngSeed=123)
train_ids = list(range(6))
test_ids = list(range(6, 10))

est.Train(train_ids, train_likelihood,
          Svi(numSteps=1500, numSamples=300, progressBar=False))
```

The first six individuals train the population model; the other four are held out. The short optimization run is for demonstration only. In research applications, inspect convergence, posterior predictive behavior, and prior sensitivity.

The conditioning example below runs a separate SVI fit for each of 20 sampled population states. It is intended to illustrate the workflow, not to be a quick smoke test; reduce `nGlobalSamples` and SVI steps when first checking that the code executes.

Predict without seeing any test observations

First define a score for the **future** measurements, at indices 3 through 7. Scoring can use held-out outcomes even though inference must not use them.

```
def future_mae(model_out, data):
    return jnp.mean(jnp.abs(model_out["prediction"][3:] - data["y"][3:]))

est.Test(test_ids, nGlobalSamples=20, nLocalZSamples=100)
before = est.ScoreTestSamples(future_mae)
print("Population-only mean future MAE:", float(jnp.mean(before)))
```

Here BET samples 20 population posterior states and generates 100 local-effect draws for each state. None of the held-out individuals' observed `y` values inform their latent effects. The score array has shape `[20, 100, 4]`: global draw, local draw, and held-out individual.

Update predictions after three observations

Next, define a test likelihood that reveals only the first three measurements. Masking is important: the remaining measurements must not affect inference.

```
def test_likelihood(global_params, local_zs, data, model_out):
    observed = jnp.arange(data["y"].shape[0]) < 3
    prediction = model_out["prediction"]
    safe_y = jnp.where(observed, data["y"], prediction)
    return {
        "y": (safe_y, dist.Normal(prediction, global_params["noise"]).mask(observed))
    }

est.Test(test_ids, TestLikelihoodFn=test_likelihood,
         Method=Svi(numSteps=800, numSamples=100, progressBar=False),
         nGlobalSamples=20)
after = est.ScoreTestSamples(future_mae)
print("Conditioned mean future MAE:", float(jnp.mean(after)))
```

BET now infers each held-out individual's local growth effect from the first three observations, conditional on each sampled population state. The score still evaluates measurements at indices 3 through 7, which were excluded from the test likelihood. This second test call replaces the previous test state; the `before` scores remain available because they were already stored in a separate variable.

Interpret the comparison

Both `before` and `after` have shape `[20, 100, 4]`. Averaging their values summarizes future absolute errors across population draws, local draws, and held-out individuals. Early measurements often improve predictions, but not necessarily in every small synthetic experiment.

The two stages answer different questions:

- **Population-only:** What can we predict for a new individual before observing their trajectory?
- **Conditioned:** What can we predict after observing that individual's first three measurements?

To examine uncertainty in trajectories rather than errors, use a score function returning `model_out["prediction"]` and summarize its posterior draw axes. The resulting intervals describe latent mean trajectories; they do not automatically include new measurement noise. For stronger evaluation, repeat across random seeds and vary the number of early observations.

The same Train → Test → Score workflow also applies to differentiable mechanistic ODE models.

Part 3: Model components and local effects

Components and function contracts

`Estimator(data, ModelFn, localZNames=None, GlobalPriorFn=None, lkjConcentration=2.0, rngSeed=None)` takes a dictionary of arrays. The first axis of each data array indexes individuals; all arrays have the same number of individuals. Remaining axes can represent times, features, or spatial coordinates.

`ModelFn(globalParams, localZs, data)` receives global parameters, one individual's local latent values, and that individual's data as dictionaries. It returns a dictionary of model outputs. BET vectorizes the function across individuals and posterior draws, so avoid Python side effects and converting traced JAX values to NumPy.

`GlobalPriorFn(dataShapes)` returns a dictionary of parameter names mapped to NumPyro distribution objects. Each shape in `dataShapes` excludes the individual axis. The global parameters are shared by the training population.

`TrainLikelihoodFn(globalParams, localZs, data, modelOut)` and `TestLikelihoodFn` return dictionaries mapping site names to `(observed_value, distribution)` tuples. A site may contain a scalar or a vector of observations. Its observation, distribution, and mask shapes must agree.

`ScoreFn(modelOut, data)` returns a scalar or array for one individual; BET vectorizes it across posterior samples and individuals.

Hierarchical local variables: standardized z-scores

Local z-scores represent entry-to-entry differences on a standardized scale. For example, `z=0` gives the model's central growth rate; positive or negative values change it according to the transformation you define. With `rate = exp(log_rate_mean + log_rate_sd * z)`, `log_rate_mean` sets the center on the log scale and `log_rate_sd` sets its spread. Before conditioning on individual observations, each z-score has a standard-normal marginal distribution.

Extend the tumor-growth example by adding treatment. Each patient now has two parameters: a **growth rate**, describing how quickly the tumor grows without treatment, and a **treatment-response parameter**, describing how strongly treatment slows that growth. Both may vary between patients. For instance, patients whose tumors grow faster might also tend to respond more strongly to treatment, or the relationship might be reversed. Assuming the parameters vary independently would miss that population-level relationship.

Set `localZNames=["growth", "response"]` to represent both sources of patient-to-patient variation as standardized local effects. BET can learn their **correlation** from the training patients. A positive correlation means patients with above-average growth effects tend also to have above-average response effects; a negative correlation means higher growth effects tend to accompany lower response effects. These are correlations between the standardized effects, not necessarily between the parameters after arbitrary transformations.

To represent this dependence while keeping each local effect marginally standard normal, BET uses a **Cholesky factor** of a correlation matrix. A correlation matrix specifies how pairs of effects vary together; its Cholesky factor is a triangular matrix that can transform independent standard-normal draws into correlated ones. BET places an LKJ prior on the correlation structure, with `lkjConcentration` controlling how strongly the prior favors correlations near zero. The inferred relationship can improve predictions for a new patient: measurements that inform their growth rate can also inform their likely treatment response, even if response measurements are limited.

The model below illustrates the parameter transformation. It is a schematic treatment model, not a validated description of tumor dynamics:

```
def model_fn(global_params, local_zs, data):
    growth = jnp.exp(global_params["log_growth_mean"] + global_params["log_growth_sd"] *
local_zs["growth"])
    response = jnp.exp(global_params["log_response_mean"] + global_params["log_response_sd"] *
local_zs["response"])
    net_rate = growth - response * data["treatment"]
    return {"prediction": data["baseline"] * jnp.exp(net_rate * data["time"])}
```

Define population means, scales, and transformations explicitly; BET does not create them automatically for each local variable.

Part 4: Inference methods

Choosing and configuring NUTS or SVI

`Nuts(numWarmup=2000, numSamples=2000, numChains=4, acceptProb=0.95, denseMass=False, medianSamples=50)` uses NumPyro's NUTS MCMC kernel with median-based initialization. `numWarmup` is the adaptation period; `numSamples` is the retained number of draws **per chain**; `numChains` controls chain count; `acceptProb` is the target acceptance probability; `denseMass` enables dense mass-matrix adaptation; and `medianSamples` controls the initialization strategy. Its reported summary includes effective sample size and R-hat. Inspect divergences, mixing, and posterior predictive adequacy rather than treating a successful run as validation.

`Svi(numSteps=10000, numSamples=2000, stepSize=0.01, numParticles=1, progressBar=True, medianSamples=50, multivariate=False)` optimizes a variational approximation with Adam and an ELBO objective. `numSteps` is the number of optimization updates; `numSamples` controls approximate posterior draws generated after optimization; `stepSize` is Adam's learning rate; `numParticles` controls Monte Carlo particles for the ELBO; and `medianSamples` controls initialization. `multivariate=False` uses `AutoNormal`; `True` uses `AutoMultivariateNormal`, which can represent posterior covariance but may be expensive in large hierarchical models.

Extending inference methods. NUTS and SVI are the built-in choices, not a closed list. Additional inference methods can be integrated through BET's Method interface; see **Writing a custom inference Method** in Part 7 and `ValidateMethod`.

Method selection. NUTS is often preferable when reliable posterior uncertainty is central and runtime is manageable. SVI can be faster for large problems, but an apparently stable loss curve does not establish calibrated uncertainty. Compare posterior predictive behavior and uncertainty against NUTS or simulated ground truth when feasible. BET supports configured Method instances; using Method classes can reconstruct previously saved configuration when available.

Part 5: Testing and scoring

Training, testing, and score dimensions

`Train(trainDataIdxs, TrainLikelihoodFn, Method, printSummary=False, rngSeed=None, saveLocalZs=True, saveModelOut=False)` estimates parameters from the selected individuals using a `Nuts(...)` or `Svi(...)` method.

`Test(testDataIdxs, TestLikelihoodFn=None, Method=None, nGlobalSamples=None, nLocalZSamples=2000, printSummary=False, rngSeed=None, saveModelOut=False)` supports population prediction for new individuals when no test likelihood is supplied, or local-effect inference conditional on a test likelihood and Method. **The current implementation requires at least one local Z for `Test`.** `nGlobalSamples=None` uses posterior medians; a positive value samples stored training draws. `nLocalZSamples` applies to the no-likelihood mode.

`ScoreTrainSamples(ScoreFn, nSamples=None, rngSeed=None)` returns `[posterior_draw, train_individual, ...score_shape]`.

`ScoreTestSamples(ScoreFn, nSamples=None, rngSeed=None)` returns `[global_draw, local_draw, test_individual, ...score_shape]`.

Interpreting test-time uncertainty

BET can represent at least two distinct sources of predictive uncertainty: uncertainty in population-level parameters and uncertainty in a new entry's local effects. The current `Test` implementation requires at least one named local Z, even though `Estimator` and `Train` can be used without local Zs. Measurement noise is described by the observation likelihood; it is not automatically included in draws of deterministic model outputs. Uncertainty in model structure is not automatically represented.

With `nGlobalSamples=None`, Test uses the posterior median for global parameters and the learned Cholesky state rather than propagating variation across the training posterior. This is a useful conditional prediction but can understate uncertainty in the population. With a positive `nGlobalSamples`, BET samples global/Cholesky states from the training posterior and performs test inference for each selected state. This can be computationally more expensive because the test inference is repeated for each selected global state.

When no test likelihood is supplied, local effects are drawn from the learned population distribution. When a likelihood is supplied, the test method updates local effects using the observations represented by that likelihood, conditional on the selected global state. If the scientific target is prediction after one or more measurements, ensure the likelihood contains only measurements available at that prediction time. Score held-out outcomes separately and do not use them for local inference.

A score array is not itself a posterior predictive interval. To report uncertainty for a prediction, retain posterior predictions or scores over draws and calculate summaries or quantiles along the correct draw axes. State explicitly whether population uncertainty was propagated and whether scores are absolute error, squared error, log score, or another metric.

Conditioning on partial observations: a concrete pattern

To predict a trajectory from its first few measurements, expose only those measurements through the test likelihood. Keep scoring separate so it can evaluate later, held-out observations.

```
def test_likelihood(global_params, local_zs, data, model_out):
    # Assumes each entry has observations on the same sorted time grid.
    n_observed = 3
    observed = jnp.arange(data["y"].shape[-1]) < n_observed
    prediction = model_out["prediction"]
    # Replace held-out values before likelihood evaluation. The mask controls
    # which finite values contribute to the log likelihood.
    safe_y = jnp.where(observed, data["y"], prediction)
    return {
        "y": (safe_y, dist.Normal(prediction, 0.2).mask(observed))
    }

est.Test(testDataIdxs, TestLikelihoodFn=test_likelihood,
         Method=Svi(numSteps=1000, numSamples=200, progressBar=False))
heldout_scores = est.ScoreTestSamples(score)
```

Here `testDataIdxs` must identify entries excluded from training. The example assumes `data["y"]` is a time series and `model_out["prediction"]` has the same shape. Adapt the mask to each entry's actual observation schedule when lengths differ. If observations are irregular or have different valid times, use a validity mask combined with the observation cutoff. Ensure the deterministic model does not use future outcomes as inputs, and make every intermediate calculation finite at padded positions.

This call conditions local effects on the first three measurements for each selected global parameter state. With `nGlobalSamples=None` (the default), BET uses posterior medians for global parameters and the Cholesky factor; it does not integrate over their posterior variation. Set `nGlobalSamples` to a positive integer to repeat local inference across sampled global posterior states. This can be expensive, because BET runs the test inference separately for each selected state.

Return values, sample axes, and state

The estimator retains the most recent training and test state. A new call to `Train` clears the prior test state. A new call to `Test` replaces the previous test state. Score functions operate on this stored state; they do not refit the model.

- `Train(...)` returns the inference method's output object. For NUTS this is NumPyro's MCMC object; for SVI it is the SVI result containing optimized parameters and losses. BET also stores the posterior samples it needs for later testing and scoring.
- `Test(...)` without a likelihood generates population predictions and returns `None`. When `saveModelOut=True`, it stores model outputs for subsequent scoring; it does not return that dictionary. With a test likelihood, it returns a list containing one inference output per selected global posterior draw. If `nGlobalSamples=None`, BET uses one median global state, so the list has one element.
- `ScoreTrainSamples(...)` returns an array with shape `[posterior_draw, training_entry, ...score_shape]`.
- `ScoreTestSamples(...)` returns an array with shape `[global_draw, local_draw, test_entry, ...score_shape]`. In the no-likelihood test mode, there is one global state if medians are used; with sampled global states, the leading axis indexes those states. The local-draw axis represents variation among newly generated or inferred local effects.
- `Save(path)` writes a compressed NPZ file and returns no meaningful value. `Save(None)` returns an in-memory nested record. `Estimator.Load(path, ModelFn)` returns the stored record as a dictionary when `ModelFn=None`; supplying a model function reconstructs an Estimator. The `ModelFn` argument must be passed explicitly.

The precise model-output shape depends on what `ModelFn` returns. Always write down expected axes before aggregating: average over posterior draws to summarize uncertainty, and over entries only when that matches the scientific question. Avoid flattening global and local draws together without considering how they were generated.

Part 6: Missing data and numerical models

Variable-length sequences and masking

Use masks when entries have different observation counts, measurements are missing, or testing should reveal only part of a trajectory. A mask distinguishes **positions stored in an array** from **observations contributing to inference**. Held-out measurements can remain in the data for scoring while being excluded from the test likelihood.

Pad each time series to a common length `T`. Store `time`, `y`, and a boolean `valid` mask as `[N, T]` arrays; padded positions are placeholders, not observations. Repeating the last valid time and measurement can be safer than inserting zeros when a solver or transformation cannot accept zero. Choose padding to satisfy the model's and solver's time-ordering requirements, and exclude padded positions from the likelihood.

```
data = {
    "time": jnp.array([[0., 1., 2.], [0., 3., 3.]]),
    "y": jnp.array([[1.0, 1.2, 1.4], [0.9, 1.1, 1.1]]),
    "valid": jnp.array([[True, True, True], [True, True, False]]),
}
```

Use a masked NumPyro distribution and replace invalid observed values with finite, safe values before log-probability evaluation:

```
def likelihood(global_params, local_zs, data, model_out):
    valid = data["valid"]
    pred = model_out["prediction"]
    safe_y = jnp.where(valid, data["y"], pred)
    return {"y": (safe_y, dist.Normal(pred, 0.2).mask(valid))}
```

The model must produce finite predictions even at padded positions. Masking cannot make an intermediate `NaN` or infinity safe; multiplying a divergent log-probability by zero does not guarantee a finite gradient.

For partial-observation testing, use a separate mask such as `test_observed = valid & (time <= cutoff)`. Apply it in the test likelihood, but retain the original validity mask for scoring held-out observations. Ensure hidden measurements cannot enter the model through inputs or preprocessing.

Numerical stability when masking

Scale observations and model outputs to reasonable ranges. Set likelihood scales according to measurement noise rather than choosing arbitrarily small values: large residuals or near-zero noise can destabilize log-probabilities and gradients. Replace missing observations with finite placeholders before masking, and check that model outputs remain finite at padded positions. Diagnose shape and gradient errors with small synthetic examples, `jax.debug.print`, and `jax.numpy.isfinite`.

Before population fitting, test the model and likelihood on one individual, then verify axes and broadcasting across individuals. Use posterior predictive checks: successful NUTS or SVI execution does not establish scientific adequacy.

JAX differential equation solvers

`ModelFn` can call a JAX-compatible ODE solver such as `jax.experimental.ode.odeint`. Use JAX arrays and differentiable operations, and return outputs at observation times. In the workflows tested for this project, `odeint` has worked more reliably with BET's batching than Diffrax. Compatibility is model-dependent, so verify gradients and numerical behavior. Diffrax is an optional external dependency, not part of BET's installation requirements.

```
from jax.experimental.ode import odeint

def rhs(state, time, rate):
    return rate * state

def model_fn(global_params, local_zs, data):
    rate = jnp.exp(global_params["log_rate"] + local_zs["rate"])
    times = data["time"]  # sorted, finite, compatible with odeint
    trajectory = odeint(rhs, data["initial"], times, rate)
    return {"prediction": trajectory}
```

Padded time grids must satisfy the solver's ordering requirements. One approach is to solve on a shared regular grid and interpolate to each individual's observation times. Check for duplicate or unsorted padded times.

Part 7: Persistence and extensibility

Saving, loading, and custom methods

`est.Save("fit.npz", saveAllData=True)` saves inference state together with the data. To restore it, call `Estimator.Load("fit.npz", ModelFn=model_fn, GlobalPriorFn=global_prior)`. If data were not saved, also supply `data=`. In either case, supply the original model and prior functions again: executable Python functions are not serialized.

Writing a custom inference Method

BET separates inference algorithms from its population and individual workflows. NUTS and SVI are built in; to integrate another backend, implement the minimum Method interface and validate it before fitting.

A configured Method instance must implement:

- `_Fit(ModelFn, rngKey, samplePrefixes)`: run inference and return a dictionary with `"output"` (backend-specific results or diagnostics) and `"samples"` (posterior or explicitly approximate-posterior draws).
- `_SaveArgs()`: return a dictionary of constructor keyword arguments so BET can reconstruct the Method from saved configuration.

The `samples` dictionary maps NumPyro sample-site names to arrays with **draws along the first axis**. Retain only sites whose names start with one of `samplePrefixes`; BET uses these names to identify global parameters, local effects, Cholesky factors, and optionally model outputs.

This minimal skeleton illustrates the interface:

```
class MyMethod:
    _printMode = None
```

```
    def __init__(self, nSamples=100):
        self.nSamples = nSamples

    def _Fit(self, ModelFn, rngKey, samplePrefixes):
        # Replace with an actual inference backend.
        rawSamples, backendOutput = run_my_inference(ModelFn, rngKey, self.nSamples)
        samples = {name: value for name, value in rawSamples.items()
                   if any(name.startswith(prefix) for prefix in samplePrefixes)}
        return {"output": backendOutput, "samples": samples}

    def _SaveArgs(self):
        return {"nSamples": self.nSamples}
```

`run_my_inference` is a placeholder, **not a BET function**. The backend must produce meaningful posterior or clearly identified approximate-posterior draws. Repeating a maximum-a-posteriori (MAP) point does not measure posterior uncertainty.

Validate the Method's structure before using it:

```
import BayesianEstimationTool as bet

method = MyMethod(nSamples=100)
bet.ValidateMethod(method)
```

`ValidateMethod` checks required callable signatures, saved constructor arguments, and printing hooks. It **does not execute** `_Fit`, inspect returned samples or shapes, or establish statistical correctness. Test those separately on small synthetic models and with posterior predictive checks.

Optional features include:

- **Diagnostics and printing:** Set `_printMode` to `"before"`, `"after"`, or `"replace"`, and implement `_Print(output, localNames, globalNames, title=None)`. With `_printMode=None`, BET uses its generic sample summary.
- **Configuration:** Accept options for initialization, optimization, sampling, and progress reporting, and return reproducible constructor options from `_SaveArgs()`.
- **Persistence:** Keep configuration reconstructible through constructor keyword arguments. The backend `output` is returned to the caller but is not automatically serialized as a live inference-engine session.
- **Inference quality:** Compare posterior summaries, predictive distributions, and computational costs against NUTS on a known model. This is behavioral testing, not structural validation.

See `NutsMethod.py` and `SviMethod.py` for complete Method implementations.

Part 8: Troubleshooting and API reference

Common mistakes and troubleshooting

Shape mismatch. Start with one entry and print array shapes before tracing. All top-level data arrays must have the same entry count, and per-entry model outputs must match likelihood observation shapes. Broadcasting can silently produce an incorrect likelihood, so verify dimensions explicitly.

Invalid entry indices. Training and test indices must be nonempty, one-dimensional collections of unique integers within the data's entry range. BET rejects duplicate, negative, and out-of-range indices; it does not silently remove duplicates or interpret negative indices as Python indexing from the end.

Local z-scores mistaken for physical parameters. A local z-score has a standard-normal marginal prior and is not measured in units of growth rate, drug sensitivity, or observation noise. Specify each transformation to physical units explicitly, and document the resulting units.

Hidden test outcomes leaking into inference. Keeping the full `y` array for scoring is safe only if the test likelihood masks unavailable outcomes and the mechanistic model does not read them. Check preprocessing, features, and masks to prevent future measurements from influencing predictions.

Missing data and NaNs. Masked observations still require finite values at points where JAX evaluates intermediate expressions. Use safe finite placeholders and keep ODE solver time grids valid even for padded observations.

Unexpectedly narrow intervals. With `nGlobalSamples=None`, BET fixes population parameters at posterior medians and does not propagate their uncertainty. Use a positive `nGlobalSamples` to include it. Approximate SVI posteriors may also underestimate uncertainty.

Poor convergence. Check observation scaling, priors, parameter identifiability, and solver gradients. For NUTS, inspect mixing and convergence diagnostics; for SVI, inspect the loss trajectory and sensitivity to initialization.

Save/Load mismatch. Restore compatible data shapes, Method configuration, model function, and global prior function. Python functions are not serialized; stored names are metadata, not executable code.

Detailed public API reference

The signatures below cover the public exports `Estimator`, `Nuts`, `Svi`, and `ValidateMethod`, together with public `Estimator` methods.

Estimator construction

```
Estimator(data, ModelFn, localZNames=None, GlobalPriorFn=None, lkjConcentration=2.0, rngSeed=None)
```

- `data`: dictionary of arrays sharing a leading entry axis.
- `ModelFn`: deterministic per-entry model function returning a dictionary.
- `localZNames`: optional list of named standardized local effects.
- `GlobalPriorFn`: optional function returning named NumPyro distributions for shared parameters.
- `lkjConcentration`: concentration of the LKJ correlation prior when multiple local effects are used.
- `rngSeed`: optional reproducibility seed.

Train

```
Train(trainDataIdxs, TrainLikelihoodFn, Method, printSummary=False, rngSeed=None, saveLocalZs=True, saveModelOut=False)
```

- `trainDataIdxs`: selected training entries.
- `TrainLikelihoodFn`: per-entry likelihood factory.
- `Method`: inference Method instance or supported Method class.
- `printSummary`: request inference summary output.
- `rngSeed`: optional call-specific seed.
- `saveLocalZs`: retain local posterior draws for later use.
- `saveModelOut`: retain generated model outputs rather than recomputing them.

Test

`Test(testDataIdxs, TestLikelihoodFn=None, Method=None, nGlobalSamples=None, nLocalZSamples=2000, printSummary=False, rngSeed=None, saveModelOut=False)`

- `testDataIdxs`: entries for population prediction or local updating.
- `TestLikelihoodFn`: omitted for unconditioned population prediction; provided for observation-conditioned inference.
- `Method`: inference Method for observation-conditioned testing; may reuse saved configuration.
- `nGlobalSamples`: `None` uses posterior median population state; a positive integer selects trained posterior states.
- `nLocalZSamples`: number of local draws in no-likelihood testing.
- `printSummary`, `rngSeed`, `saveModelOut`: reporting, reproducibility, and output retention.

Scoring

`ScoreTrainSamples(ScoreFn, nSamples=None, rngSeed=None)` scores training posterior predictions. Its output axes are posterior draw, training entry, then any score dimensions. If the model has local Zs, training must retain either local posterior draws (`saveLocalZs=True`, the default) or model outputs (`saveModelOut=True`); otherwise training scores cannot be reconstructed.

`ScoreTestSamples(ScoreFn, nSamples=None, rngSeed=None)` scores the most recent test state. Its output axes are global draw, local draw, test entry, then any score dimensions. Check what each axis represents before aggregating uncertainty.

Models without local effects: `Estimator` and `Train` support models without local Zs, but the current `Test` implementation rejects them (`Test() requires at least one local Z`). Setting `saveModelOut=True` does not bypass this restriction. Consequently, `ScoreTestSamples` cannot be used for such models through the `Test` workflow.

Persistence

`Save(fileName, saveTrainLocalZs=True, saveTestLocalZs=True, saveTrainModelOut=False, saveTestModelOut=False, saveAllData=False)` writes a compressed state file; with `fileName=None`, it returns a record. The flags control whether local draws, model outputs, and data are included. Saving data improves portability but increases file size and may create data-sensitivity obligations.

`Estimator.Load(fileName, ModelFn, GlobalPriorFn=None, data=None)` reconstructs an estimator using the supplied functions. Provide `data` if it was not saved. Passing `ModelFn=None` returns the saved record without reconstructing an executable estimator.

`ValidateMethod(method)` checks a custom Method's required interface; see **Writing a custom inference Method** in Part 7 for implementation details. Public exports are defined in package `__init__.py`.

Part 9: Verification

Verification and further reading

The repository's `TESTING.md` explains how to run automated tests, while Part 7 documents the custom inference interface. The included `Examples/` directory contains NUTS, SVI, and Save/Load scripts. The one-page `docs/Cheatsheet.md` is a compact companion, not a replacement for this manual.

Smoke and integration tests show that supported workflows execute; they do not establish identifiability, credible-interval calibration, or scientific validity. For research claims, add application-specific simulation-based recovery and posterior predictive

checks.

Test strategy

The regression suite covers NUTS, SVI, scoring, validation, and Save/Load. Scientific validation also calls for synthetic hierarchical parameter recovery, transfer to new individuals with different observation counts, and numerical equivalence after persistence. See `TESTING.md` for test commands.